

Validación de Unicidad en JPA con Spring Boot y Jakarta EE

Para garantizar la *unicidad* de un campo o un conjunto de campos en una entidad JPA con **Spring Boot** y **Jakarta EE**, tienes varias opciones, aunque las anotaciones de validación estándar no incluyen una verificación directa para unicidad. A continuación te explico cómo se puede manejar la unicidad a nivel de JPA y validación.

1. Unicidad a nivel de base de datos: Anotación `@UniqueConstraint` (JPA)

Para garantizar que un campo o un conjunto de campos sea único a nivel de base de datos, se utiliza la anotación `@UniqueConstraint` en la clase de la entidad JPA. Esto se asegura de que la base de datos aplique una restricción de unicidad en la columna o combinación de columnas especificadas.

Ejemplo: Unicidad de una columna (email):

```
@Entity
@Table(name = "usuarios", uniqueConstraints = {
    @UniqueConstraint(columnNames = "email")
})
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false, unique = true)
    private String email;

    // Otros campos y métodos
}
```

En este ejemplo, la columna **email** tendrá una restricción de unicidad, tanto a nivel de JPA como en la base de datos. La clave aquí es la combinación de `@Table` con `@UniqueConstraint`, que permite especificar una o más columnas únicas.

Ejemplo: Unicidad de un conjunto de columnas (email y nombre):

```
@Entity
@Table(name = "usuarios", uniqueConstraints = {
    @UniqueConstraint(columnNames = {"email", "nombre"})
})
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
```

```

    @Column(nullable = false)
    private String email;

    @Column(nullable = false)
    private String nombre;

    // Otros campos y métodos
}

```

En este caso, la combinación de **email** y **nombre** debe ser única. No podrá haber dos filas en la base de datos con la misma combinación de valores para ambos campos.

2. Validación de unicidad a nivel de aplicación: Personalizando validaciones

Para validar la unicidad a nivel de aplicación antes de que los datos lleguen a la base de datos, puedes crear una **validación personalizada** con **Jakarta Bean Validation**.

Pasos para crear una validación personalizada de unicidad:

1. **Crear la anotación personalizada** para validar la unicidad de un campo o conjunto de campos.
2. **Crear el validador** que verifica si el valor ya existe en la base de datos.
3. **Usar la anotación en la entidad** para validar antes de que se inserten los datos.

Paso 1: Crear la anotación personalizada @UniqueValue

```

import jakarta.validation.Constraint;
import jakarta.validation.Payload;
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Constraint(validatedBy = UniqueValueValidator.class)
@Target({ ElementType.FIELD })
@Retention(RetentionPolicy.RUNTIME)
public @interface UniqueValue {
    String message() default "El valor debe ser único";
    Class<?>[] groups() default {};
    Class<? extends Payload>[] payload() default {};
    Class<?> entity(); // Entidad a validar
    String fieldName(); // Nombre del campo en la entidad
}

```

Paso 2: Crear el validador UniqueValueValidator

El validador verifica en la base de datos si el valor del campo ya existe. Utiliza el **EntityManager** de JPA para consultar la existencia del valor.

```
import jakarta.validation.ConstraintValidator;
import jakarta.validation.ConstraintValidatorContext;
import org.springframework.beans.factory.annotation.Autowired;

import javax.persistence.EntityManager;
import javax.persistence.PersistenceContext;
import javax.persistence.Query;

public class UniqueValueValidator implements
    ConstraintValidator<UniqueValue, Object> {

    private String fieldName;
    private Class<?> entityClass;

    @PersistenceContext
    private EntityManager entityManager;

    @Override
    public void initialize(UniqueValue constraintAnnotation) {
        this.fieldName = constraintAnnotation.fieldName();
        this.entityClass = constraintAnnotation.entity();
    }

    @Override
    public boolean isValid(Object value, ConstraintValidatorContext context) {
        if (value == null) {
            return true; // Valor nulo lo manejamos con @NotNull
        }

        Query query = entityManager.createQuery(
            "SELECT COUNT(e) FROM " +
            entityClass.getName()
            + " e WHERE e." + fieldName + " = :value"
        );
        query.setParameter("value", value);

        Long count = (Long) query.getSingleResult();
        // Retorna verdadero si no existe, es decir, es único
        return count == 0;
    }
}
```

Paso 3: Usar la validación en la entidad

Aplicamos la anotación `@UniqueValue` en la entidad donde queremos garantizar la unicidad. En este caso, validamos que el **email** sea único en la entidad `Usuario`.

```
@Entity
public class Usuario {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @UniqueValue(entity = Usuario.class, fieldName = "email",
                 message = "El email ya está registrado")
    @Column(nullable = false)
    private String email;

    // Otros campos y métodos
}
```

3. Combinación de validación personalizada y restricción de base de datos

Aunque puedes implementar la validación a nivel de aplicación, **la mejor práctica es siempre combinarla con la validación a nivel de base de datos** utilizando el enfoque de `@UniqueConstraint` en la tabla JPA, ya que la base de datos es el último nivel de control de los datos. Esto asegura que incluso si por alguna razón no se valida en la aplicación, la base de datos mantendrá la integridad.

Conclusión:

Para garantizar la unicidad de campos en tu aplicación de **Spring Boot** con **Jakarta EE**, debes combinar validaciones a nivel de base de datos mediante `@UniqueConstraint` con validaciones a nivel de aplicación cuando sea necesario mediante un validador personalizado. Esto asegura que el dato sea verificado tanto en la aplicación como en la base de datos, evitando duplicados no deseados.

CascadeType en Spring Boot y JPA

En **Spring Boot** (y más generalmente en JPA, que es la tecnología subyacente), el atributo `'cascade'` de las relaciones entre entidades define cómo se propagan las operaciones realizadas en una entidad principal a sus entidades relacionadas. Esto se utiliza junto con las anotaciones de relaciones como `'@OneToOne'`, `'@OneToMany'`, `'@ManyToOne'`, o `'@ManyToMany'`.

El `'CascadeType'` se refiere a los tipos de acciones que se aplicarán en cascada desde la entidad principal a las entidades asociadas. Estas acciones pueden incluir operaciones como **persistir**, **actualizar**, **eliminar**, etc.

Tipos de 'CascadeType':

1. 'CascadeType.ALL'

- Aplica **todas** las operaciones en cascada (incluso eliminar, persistir, etc.) a la entidad relacionada.
- Significa que todas las operaciones (persist, merge, remove, refresh, detach) que se realicen en la entidad principal también se aplicarán a las entidades relacionadas.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.ALL)
private List<Prestamo> prestamos;
```

En este caso, si creas, actualizas o eliminas un 'Usuario', todas esas operaciones también afectarán a los 'Prestamos' relacionados.

2. 'CascadeType.PERSIST'

- Solo hace que la operación de **persistencia** (guardar una entidad nueva) se propague en cascada.
- Cuando se llama a 'EntityManager.persist()' o a un método de repositorio que guarda la entidad principal, también persistirá las entidades relacionadas si estas no existían previamente.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.PERSIST)
private List<Prestamo> prestamos;
```

En este ejemplo, cuando guardas un 'Usuario' nuevo, también se guardarán automáticamente los 'Prestamos' relacionados que no existían antes, pero no se actualizarán o eliminarán los existentes.

3. 'CascadeType.MERGE'

- Propaga la operación de **actualización** (merge) desde la entidad principal hacia las entidades relacionadas.
- Cuando se actualiza la entidad principal, las entidades relacionadas también se actualizarán si están incluidas en la relación.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.MERGE)
private List<Prestamo> prestamos;
```

En este caso, si actualizas un 'Usuario', los 'Prestamos' asociados también se actualizarán automáticamente.

4. 'CascadeType.REMOVE'

- Propaga la operación de **eliminación** (remove) en cascada. Cuando se elimina la entidad principal, también se eliminarán las entidades relacionadas.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.REMOVE)
private List<Prestamo> prestamos;
```

Si eliminas un 'Usuario', automáticamente se eliminarán todos los 'Prestamos' relacionados.

5. 'CascadeType.REFRESH'

- Aplica la operación de **refrescar** en cascada. Cuando se llama a 'EntityManager.refresh()' en la entidad principal, también se recargan las entidades relacionadas desde la base de datos,

desechando cualquier cambio local.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.REFRESH)
private List<Prestamo> prestamos;
```

Si actualizas un 'Usuario' en la base de datos y luego lo refrescas, también se actualizarán sus 'Prestamos' asociados desde la base de datos.

6. 'CascadeType.DETACH'

- Propaga la operación de **detach** en cascada. Cuando se desasocia la entidad principal del **EntityManager** (es decir, deja de ser gestionada), también se desasocian las entidades relacionadas.

```
@OneToMany(mappedBy = "usuario", cascade = CascadeType.DETACH)
private List<Prestamo> prestamos;
```

Si se llama a 'EntityManager.detach()' sobre un 'Usuario', los 'Prestamos' asociados también dejarán de ser gestionados por el 'EntityManager'.

Combinaciones de 'CascadeType':

También es posible combinar varios tipos de 'CascadeType' si no deseas aplicar **todas** las operaciones en cascada, pero sí un subconjunto de ellas. Esto se puede hacer utilizando un array de tipos de cascada.

```
@OneToMany(mappedBy = "usuario",
cascade = {CascadeType.PERSIST, CascadeType.MERGE})
private List<Prestamo> prestamos;
```

En este caso, solo se aplicarán las operaciones de **persistir** y **actualizar** en cascada. Las operaciones de eliminar o refrescar no se propagarán a las entidades relacionadas.

Resumen de 'CascadeType':

Tipo	Descripción
'CascadeType.ALL'	Aplica todas las operaciones (persistir, actualizar, eliminar, etc.) en cascada.
'CascadeType.PERSIST'	Solo propaga la operación de guardar (persistir) a las entidades relacionadas.
'CascadeType.MERGE'	Solo propaga la operación de actualización (merge) a las entidades relacionadas.
'CascadeType.REMOVE'	Propaga la operación de eliminar en cascada, eliminando también las entidades relacionadas.
'CascadeType.REFRESH'	Propaga la operación de refrescar para recargar las entidades desde la base de datos.
'CascadeType.DETACH'	Propaga la operación de desasociar para que las entidades dejen de estar gestionadas.

Buenas prácticas con 'CascadeType':

- **Uso de 'CascadeType.ALL'**: Debe utilizarse con cuidado, ya que puede llevar a un comportamiento no deseado si no controlas qué operaciones se propagan. Es útil en relaciones donde quieres aplicar todas las operaciones en cascada, como en relaciones padre-hijo.
- **'CascadeType.REMOVE'**: Úsalo solo cuando estés seguro de que quieres eliminar automáticamente las entidades relacionadas cuando se elimine la entidad principal. De lo contrario, podrías perder datos no intencionadamente.

- **Combinaciones:** En la mayoría de los casos, se usa una combinación de '**PERSIST**' y '**MERGE**', ya que estas cubren la mayoría de las operaciones de crear y actualizar sin tocar eliminaciones o refrescos.